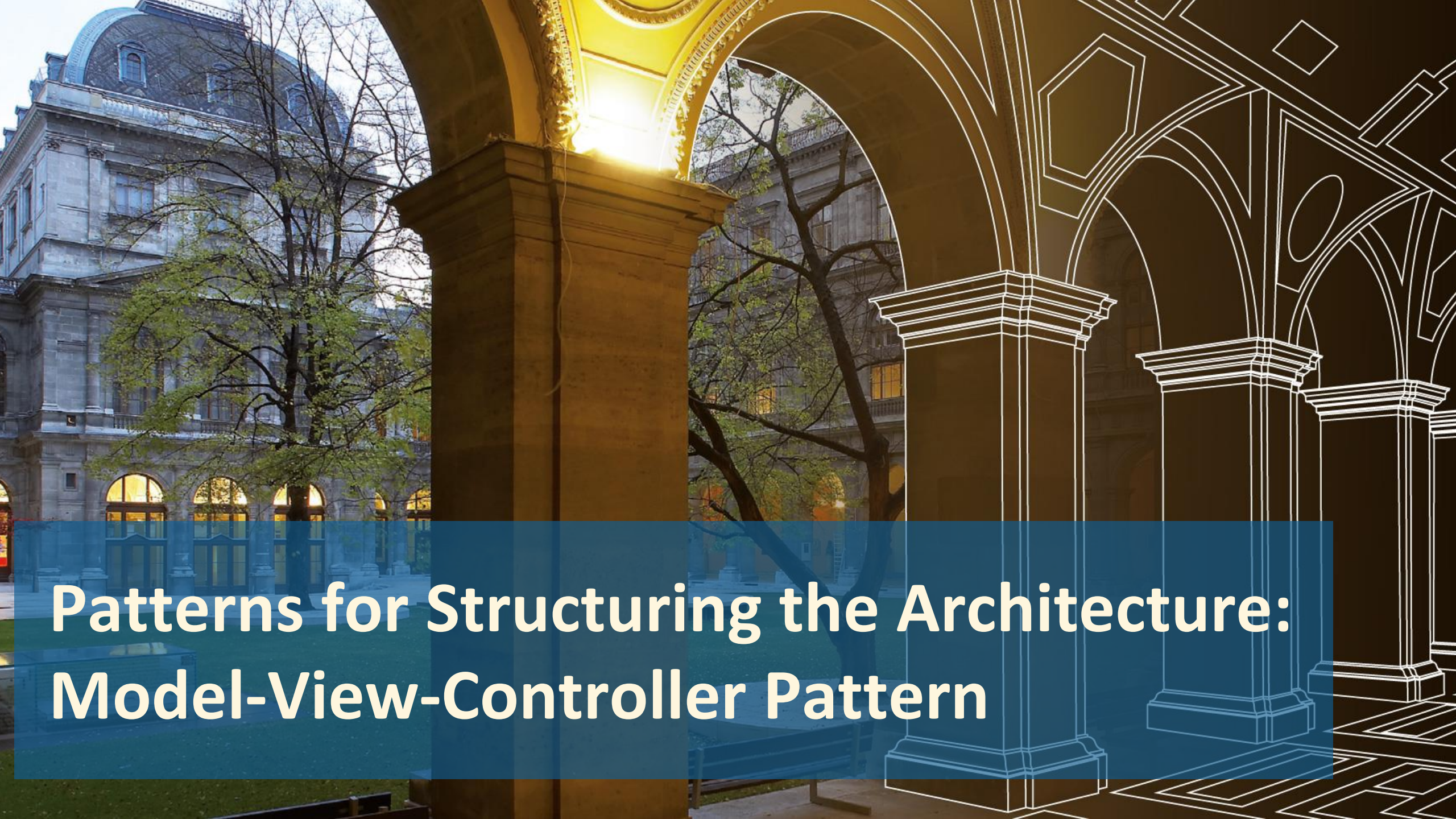




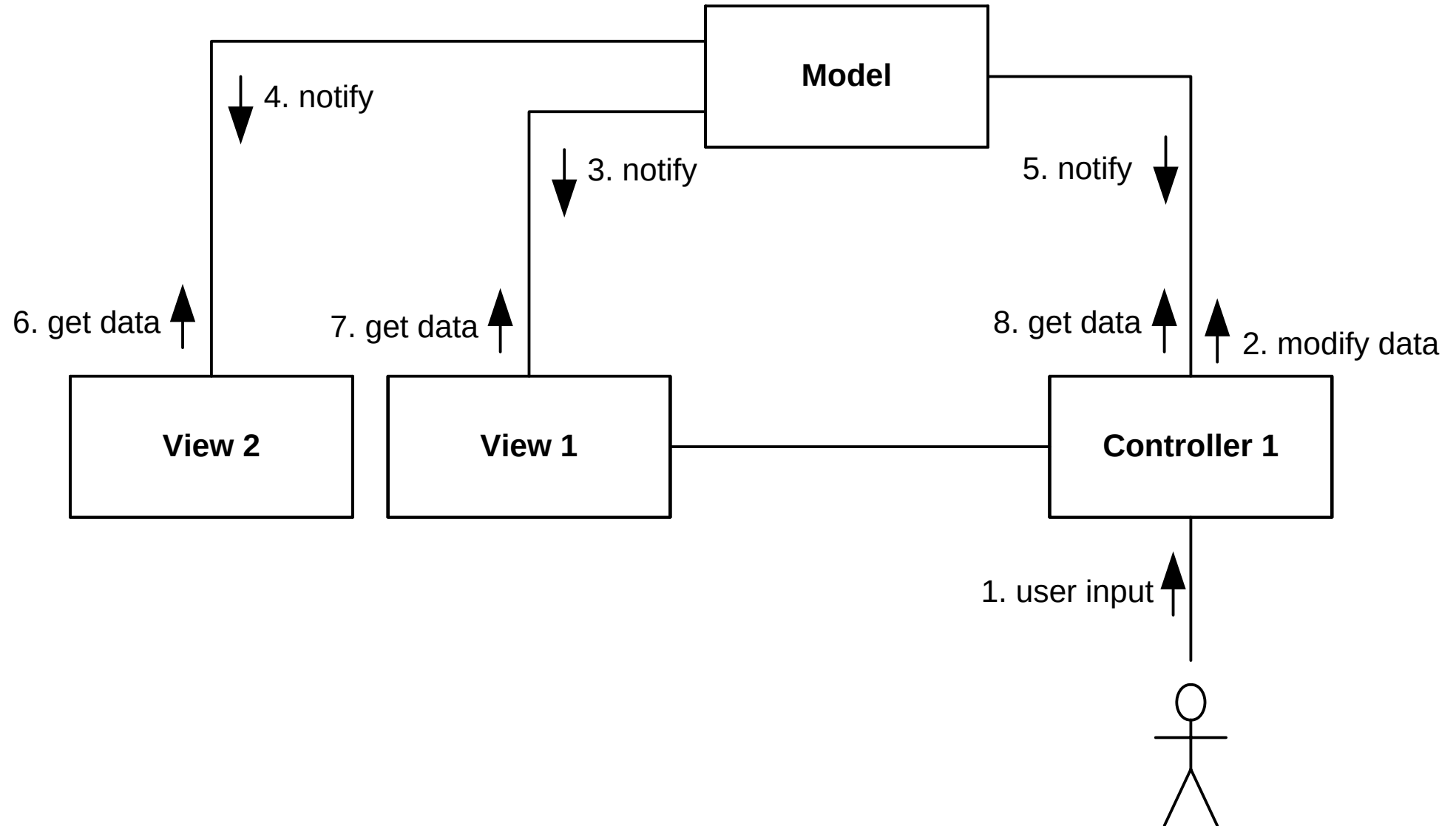
Patterns for Structuring the Architecture: Model-View-Controller Pattern

Model-View-Controller (MVC) Pattern

The Model-View-Controller (MVC) pattern divides an interactive application into three parts: The model contains the core functionality and data. Views display information to the user. Controllers handle user input.

Views and controllers together comprise the user interface. A change propagation mechanism ensures consistency between the user interface and the model.

MVC: Schematic Example



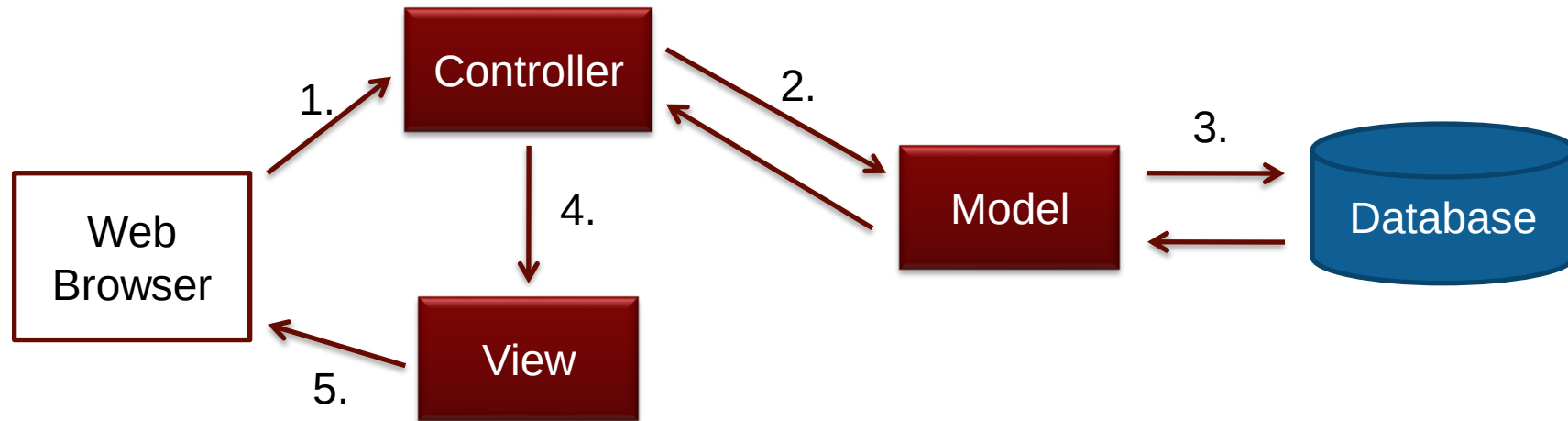
Model-View-Controller Pattern: Problem

- A system may offer **multiple user interfaces**
 - Each user interface depicts all or part of some application data
- **Changes to the data** should be automatically and flexibly reflected to all the different user interfaces
- It should be also possible to **easily modify any one of the user interfaces**, without affecting the application logic associated with the data

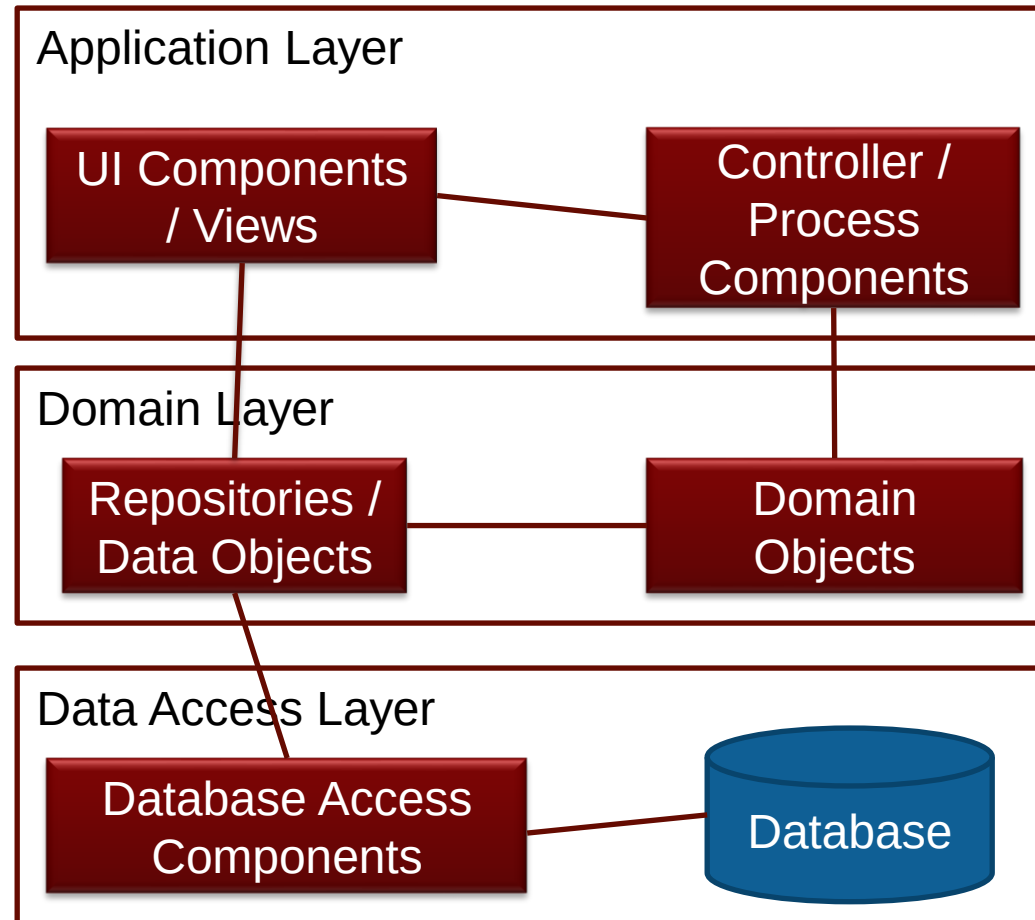
Model-View-Controller Pattern: Solution

- The system is divided into three different parts:
 - a **Model** that encapsulates some application data and the logic that manipulates that data, independently of the user interfaces
 - one or multiple **Views** that display a specific portion of the data to the user
 - a **Controller** associated with each View that receives user input and translates it into a request to the Model
- The users interact strictly through the Views and their Controllers, independently of the Model
- The model notifies all user interfaces about updates

Ruby on Rails (and Many Similar Frameworks) Are Based on an MVC Architecture: Schematic View



Such MVC Web Applications Often Also Follow a Layers Architecture



MVC: Benefits

- **Multiple views** of the same model
- **Synchronized views**
- **'Pluggable'** views and controllers
- **Exchangeability of 'look and feel'**

MVC: Downsides

- **Increased complexity**
- Potential for an **excessive number of updates**
- **Intimate connection** between view and controller
- Close **coupling** of views and controllers to a model
- **Inefficiency of data access** in view

Summary: Model-View-Controller Pattern

- Model-View-Controller is a pattern for structuring interactive applications that separates an application into:
 - The Model contains core functionality and data
 - Views display data to the user
 - Controllers handle user input
- MVC allows multiple user interfaces to reflect changes in data automatically and flexibly, enabling easy modification without affecting application logic
- However, it is more complex than simpler UI structuring approaches

Relevant Literature and Sources

- F.Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, “Pattern-Oriented Software Architecture Volume 1 - A System of Patterns”, John Wiley and Sons (1996)
- F.Buschmann, K. Henney, D.C. Schmidt, ,Pattern-Oriented Software Architecture Volume 4: A Pattern Language for Distributed Computing`, John Wiley and Sons (2007)
- Martin Fowler. Patterns of Enterprise Application Architecture. Addison Wesley, 2002.
See also: <http://martinfowler.com/eaCatalog/index.html>

